

Adaptive Inference Network

A research proposal for an evolving compute coordination protocol

Kilian Codaccioni | Version 0.2 | 30 September 2026

Status: research draft with a limited local prototype. Not a deployed network, audited protocol, token offering or claim of autonomous intelligence.

Abstract

Adaptive Inference Network (AIN) investigates whether independent compute providers can cooperate to serve model inference while maintaining an inspectable record of execution commitments and protocol changes. Its central hypothesis is that network growth can improve useful capacity, placement and resilience when admission, measurement and incentives are designed around completed work rather than node count. Better model quality is a separate outcome requiring licensed data, training, evaluation and explicit release decisions.

The proposed architecture separates off-chain computation, operational measurement, model governance and blockchain settlement. Adaptation is a controlled loop: measure, propose, compare, approve, stage and, when necessary, roll back. Neither a model nor an adaptive scheduler may unilaterally rewrite consensus rules, alter balances or promote its own evaluation result. The first research phase uses an existing EVM environment; a dedicated chain is a conditional future option, not a prerequisite.

The current implementation benchmarks a trained MiniLM sentence encoder split across two local CPU processes, with ordered hash receipts on a local EVM and a full-model reference. It also retains the initial untrained fixture. Remote networking, adaptive scheduling, economic rewards, governance and cryptographic verification of inference are not implemented. The measured benchmark, not a deployable DePIN protocol, is the selected deliverable.

Research question

Can a versioned coordination protocol improve **successful, quality-constrained inference per unit of total cost** as independently operated resources join, without making correctness or governance depend on a self-reported performance score?

Reading guide

Sections 1-3 define scope and architecture. Sections 4-6 describe adaptation, verification and economics. Sections 7-9 specify governance, experiments and delivery gates. Section 10 inventories existing evidence and references.

Repository: <https://github.com/suprkco/Blockchain-adaptive-inference-network>

0. Strategic choice and measured result

Version 0.2 selects a benchmarking deliverable. The purpose is to quantify the cost of splitting a trained model and attaching a receipt ledger, not to sell an economically or cryptographically verified compute network. MiniLM is a sentence encoder useful for retrieval; it is not a generative large language model [6].

single-short: local forward 10.50 ms; two-worker path 12.41 ms; instrumented tensor codec 0.66 ms; ledger calls 4.80 ms; measured split-plus-ledger path 17.90 ms. Values are p50 over 20 warm observations.

batch-four: local forward 21.02 ms; two-worker path 23.47 ms; instrumented tensor codec 0.70 ms; ledger calls 5.11 ms; measured split-plus-ledger path 29.97 ms. Values are p50 over 20 warm observations.

batch-two-long: local forward 66.56 ms; two-worker path 72.21 ms; instrumented tensor codec 0.83 ms; ledger calls 4.92 ms; measured split-plus-ledger path 78.14 ms. Values are p50 over 20 warm observations.

The split path was slower than local computation for all three workloads. All measured embeddings matched the reference exactly in this environment. The codec is already included in split time. End-to-end also includes the Node/Python controller bridge, and medians are not additive. These are local CPU and automining EVM measurements, not WAN or public-chain consensus results. Tokenization, startup and model download are excluded from warm intervals and documented separately.

The reference uses the library's full forward, including its unused BERT pooler; the split path omits that unused pooler. No output-quality benchmark, concurrent-throughput study, memory saving or independent-session statistical confidence is claimed.

Economic verification is deferred for a technical reason: a conflicting hash identifies disagreement, not the correct party. Slashing on that fact alone would let a malicious challenger punish an honest provider. A real optimistic protocol needs a validity rule, available evidence and an adjudication mechanism. Simulated TEE signatures and mock ZK proofs would not close this gap either.

The measurable next extension is controlled remote execution, followed by an independently specified verification experiment. The following sections preserve the longer-term network proposal with those dependencies explicit.

1. What it means for the network to improve

More participants do not automatically produce faster inference. A sequential model path can become slower when a new stage adds network latency. New capacity is useful only when the scheduler can place compatible workloads on it while preserving service constraints. Idle or unreliable nodes should not count as progress.

AIN separates four kinds of improvement. **Capacity** means more jobs can complete within a declared service envelope. **Operational efficiency** means better routing, batching, placement, recovery or replication for the same model and workload. **Model quality** means a candidate checkpoint passes a separate evaluation process. **Protocol quality** means a versioned rule change improves a defined property without violating established invariants. None implies the others.

An illustrative objective is $\text{minimize } p95 \text{ latency} + a * \text{cost_per_success} + b * \text{failure_rate}$, subject to minimum quality, privacy class and provider-concentration limits. Coefficients and units must be fixed before comparison; combining arbitrary scores after seeing results invites metric gaming. The initial evaluation should publish a Pareto comparison rather than hide tradeoffs behind one universal ranking.

The intended first users are operators running a small private federation with a model that exceeds one worker's memory, or independent providers supplying capacity for asynchronous jobs. Consumer chat across arbitrary internet peers is a harder later target. The design does not assume every request uses every node. Some nodes replicate stages; others run independent requests or remain outside a path because their connectivity is unsuitable.

Non-goals for version 0.2

- Inventing a new consensus mechanism or equating inference with proof of work.
- Guaranteeing monotonic quality gains as node count or usage increases.
- Training on users' prompts by default or treating participation as consent.
- Launching a token, selling capacity guarantees or predicting token value.
- Calling an untrained numeric fixture a large language model.

Falsifiable thesis: additional eligible providers should improve at least one predeclared service metric without breaching the others. If this does not hold under measured workloads, the claimed network effect is absent for that deployment.

2. Prior work and the role of a blockchain

Petals demonstrates serving trained model blocks over distributed participants and studies practical communication and failure constraints [1]. It motivates separating model execution from any ledger. Its results do not transfer to this prototype, which uses local IPC and a small trained encoder, plus an earlier untrained fixture.

Gensyn's Verde studies verification through refereed delegation and reproducible machine-learning execution [2]. It highlights why heterogeneous floating-point execution complicates disputes. AIN has no equivalent verification mechanism and must not describe its hashes as proofs.

Bittensor describes a network of subnets where miners provide services and validators score them [3]. Such incentive coordination is relevant background; an agreed score is not, by itself, evidence that a particular inference was executed correctly. DiLoCo studies communication-efficient distributed training [4]. Training is relevant to future model releases but is not a side effect of pooling inference capacity.

AIN's proposed contribution is an **inspectable adaptation and release process**, with explicit separation between service measurements, computation evidence and authority to change protocol state. This is a design hypothesis, not a demonstrated research novelty. Existing systems and published mechanisms must be compared before making a novelty claim.

Ledger or ordinary database?

A shared ledger is potentially useful when independently controlled providers dispute assignments, receipts or settlement and do not accept one operator's database as authoritative. It cannot solve dishonest computation, data availability or numerical reproducibility by merely storing a hash. Ethereum's oracle model makes this off-chain trust boundary explicit [5].

For a trusted cluster, a scheduler plus an append-only database is the mandatory baseline and may be the better system. For the research federation, start with contracts on an existing EVM test environment. Consider a dedicated appchain or rollup only after measuring sustained demand, ledger overhead, security dependencies and an actual need for specialized rules. No consensus or data-availability design for a new chain is specified here.

Decision rule: retain the ledger only if independently controlled participants need its shared state and its benefits exceed operational and verification costs. Do not create a blockchain solely to label the compute network decentralized.

3. Proposed architecture and job lifecycle

The **execution layer** serves model stages off-chain. The **measurement layer** collects completed-job evidence and independent probes. The **coordination layer** selects compatible providers using a pinned policy version. The **ledger layer** records commitments, authorized state transitions and later settlement rules. The **release layer** evaluates changes to models, schedulers and protocol rules separately.

Initial roles are requester, compute provider, coordinator, evaluator and release approver. Several roles may belong to one operator in the prototype; that is a centralization assumption, not decentralization by naming. Future federations must disclose common ownership and failure domains. Blockchain validators order transactions; they are not automatically inference evaluators.

Proposed job manifest

Each job binds a schema version, chain/registry identity, unique nonce, requester, model and tokenizer revisions, stage layout, tensor format, runtime/precision profile, generation configuration, deadline, maximum cost, privacy class and verification policy. Commitments require canonical serialization and domain separation. The existing contract stores only a subset: job ID, model hash, input hash, ordered workers and deadline.

An admitted job reserves a compatible route. Workers exchange authenticated, bounded activation messages carrying the job and stage identifiers. A receipt binds the input/output commitments, assigned provider and model revision. Large artifacts remain outside the ledger with declared retention and retrieval rules. A hash without retrievable data cannot support a challenge. Prompts and activations remain visible to authorized processors; off-chain storage does not make them confidential.

The proposed lifecycle is admitted -> assigned -> executing -> completed_pending -> accepted -> settled, with separate expired, cancelled and disputed paths. These are future states. The current contract supports creation, ordered receipt submission and cancellation after expiration; it has no acceptance, challenge or payment mechanism.

For a sequential path, a rough latency model is $T = \sum(\text{compute}_i) + \sum(\text{RTT}_i + \text{bytes}_i / \text{bandwidth}_i) + \text{queueing} + \text{recovery}$. Autoregressive decoding repeats communication across generated tokens, while prefill has different tensor sizes. Production scheduling must measure both. Settlement should not be on the per-token critical path; batched receipts need their own availability and dispute design.

4. Adaptation through bounded feedback

Loop A: capacity and scheduling

A new provider first runs public calibration workloads for a specific model, precision and runtime profile. Record latency distributions, memory limits, failure rates and observed transfer costs. Self-reported GPU identity or nominal throughput is not sufficient admission evidence. Eligibility is scoped to a workload class and expires when observations become stale.

The first scheduler should be deterministic and explainable: choose compatible stages under memory and deadline constraints, prefer measured locality, then compare against round-robin and a static placement baseline. An adaptive policy may use time-decayed observations and a small, bounded exploration budget. Run it in shadow mode first: log its proposed route without changing production execution.

Only selected routes normally produce feedback. This is partial feedback, so directly applying a full-information Hedge algorithm would be unjustified unless unselected routes are also measured. A contextual-bandit or conservative exploration design is a research option, not an implemented property. No theoretical regret bound is claimed for this network.

Loop B: model releases

More inference does not change model weights. Candidate checkpoints require an authorized data source, reproducible training recipe, pinned lineage and independent evaluation. Aggregate operational logs are not a license to reuse prompt content. Contributors may submit model or adapter candidates, but evaluation must include held-out tasks, safety/reliability cases and compatibility tests. Quality and efficiency should be reported separately.

Model promotion does not automatically force adoption. Requesters pin a model version or explicitly opt into a release channel. Adapters must identify their base model; unrelated weights must not be averaged as though they shared a compatible training trajectory. Training mechanisms such as DiLoCo are possible future research inputs, not implemented features [4].

Loop C: protocol releases

The network may propose revised admission, routing or evaluation rules from observed evidence. Such proposals pass a governed release process. The AI system is not authorized to change its own verifier, relax its acceptance thresholds or replace consensus rules. Operational learning remains bounded by a versioned policy envelope.

5. Correctness, availability and adversarial behavior

The threat model includes fabricated activations, replayed receipts, model substitution, selective failures, evaluator collusion, Sybil identities, withheld traces and strategic performance reporting. It also includes honest disagreements caused by different kernels, precision, hardware or sampling configuration. A cryptographic hash binds bytes; it does not establish that those bytes are a correct model output.

Current assurance: trusted local workers, shape/range validation, process timeouts, ordered on-chain statements and comparison with a full local trained-encoder reference. The latter provides a test reference but eliminates any compute-saving benefit. An explicit contract test accepts a false output hash from an authorized worker, documenting the gap.

Proposed research ladder: first reproduce a trained model across a controlled numerical profile; next introduce independent audit jobs with disclosed trust assumptions; then assess reproducible replay or a narrowly specified dispute protocol. Redundant agreement can still be collusive. Secure enclaves shift trust to hardware and attestation. Cryptographic proofs require a supported execution representation and measured overhead. None is a free substitute for a threat model.

If an audit is sampled with probability q and detects a dishonest result with conditional probability d , per-attempt detection is $q \cdot d$. Under independent sampling and unchanged behavior, detection over m dishonest attempts is $1 - (1 - q \cdot d)^m$. These are illustrative probability identities, not measured security levels. Selective cheating, leaked probes, correlated auditors and Sybils invalidate simple assumptions. Audit selection must be unpredictable until a provider commits, and the sampler and randomness source themselves need analysis.

For disputes, evidence needs canonical bytes, signed context, a retention window, retrievability, an adjudication rule and bounded resolution time. A scalar distance threshold for tensors can be useful for compatibility tests but does not prove semantic correctness. Missing data should not be relabeled numerical disagreement. In the first federation, ambiguous cases should fail closed for settlement and be reviewed manually; no autonomous slashing is justified.

Privacy remains separate: low-entropy hashes can be guessed and activations may reveal inputs. Start with synthetic or public workloads. Sensitive serving requires a separately validated confidentiality design.

6. Contribution, reputation and sustainable incentives

The prototype has no token, payments or stake. The first experiments should use accounting credits without redeemable value. This permits studying scheduling and adversarial behavior without assuming that a speculative asset will fund operations or establish trust.

A future pricing experiment should distinguish useful accepted work, reserved capacity, redundancy and independent audits. A proposed per-job cost envelope is $\text{provider compute} + \text{transfer} + \text{audit} + \text{ledger} + \text{recovery allowance}$. Use measured end-to-end cost per successful quality-constrained job. Counting FLOPs, generated tokens or signed receipts alone would reward waste or unverifiable claims.

Provider quotes could specify a maximum charge and a workload envelope. Requesters should authorize a cap before dispatch. The ledger could later record accepted invoices, but payment logic requires a dispute rule, data availability, denomination, finality assumptions and separate security review. No native token is needed to evaluate this hypothesis.

Reputation should be workload-specific, time-decayed and backed by independently checked observations. Distinguish recent availability from correctness evidence. Do not silently use historical reputation to accept unverified outputs. New-provider exploration is necessary to avoid incumbent lock-in, but identity creation must not multiply rewards or voting power. Rate limits or allowlisting are honest first-phase assumptions; a permissionless Sybil-resistant design remains open.

For a failure-free sequential route with independent provider success probabilities p_i , route success is approximately $\text{product}(p_i)$. This simple model explains why adding stages can reduce reliability. Shared power, hosting or ownership creates correlated failures, so replica count alone overstates resilience. Evaluate geographic and administrative failure domains explicitly.

An early viable network may favor asynchronous batch workloads and geographically close inference clusters. Broadly pooling memory can expand the set of models that fit, without improving interactive latency. A credible result may therefore be increased feasibility or availability rather than speed. Report the benefit actually measured, and reject incentive mechanisms whose overhead consumes that benefit.

7. Controlled evolution and governance

AIN proposes a release workflow, not an autonomous self-modifying chain. Each proposal identifies an affected component, immutable source/artifact hashes, a migration plan, comparison baselines, acceptance gates, evaluators and rollback conditions. Model weights, scheduler policies and contract versions are separate release objects.

The proposed progression is draft -> reproducible experiment -> independent review -> shadow evaluation -> limited canary -> approved release. Any gate may reject the proposal. Experiments must record unsuccessful candidates so that repeated search on one benchmark is not mistaken for independent validation. Rotating private holdouts require evaluator controls; publishing a benchmark once does not make it resistant to future overfitting.

The first federation should use a disclosed human approval policy with multiple independent reviewers. A multisignature or timelock is a future mechanism to implement, not an existing protection. The present registry is not upgradeable. New contract versions would be separate deployments with explicit migration; its historical receipts cannot be edited by a rollback.

Rollbacks mean redirecting **new** jobs to an earlier model, scheduler or contract version. In-flight jobs remain pinned to their original manifest or are explicitly cancelled under predeclared rules. Finalized transfers or ledger history cannot simply be undone. Model versioning should preserve reproducibility and allow requesters to decline an upgrade.

Immutable design constraints should include: no unapproved expansion of data use; no settlement from unaccepted evidence; no change to a job's pinned semantics after acceptance; and no unilateral evaluator promotion of its own candidate. A future governance system must also address reviewer capture, emergency authority, disclosure of conflicts, revocation and participant exit.

Adaptive routing should use off-chain telemetry within published bounds. Promoting that telemetry into consensus-critical decisions would require deterministic validation and an adversarial analysis. Availability, economic settlement and chain finality should not depend on a continuously changing opaque model. The network may learn how to operate; its authority to change rules remains explicit and inspectable.

8. Evaluation plan and stop criteria

The following are **proposed experiments, not results**. Freeze model revision, runtime, tokenizer, prompts, generation configuration, workload mix and measurement method before each comparison. Separate prefill from decoding and include warm-up, cache state and concurrency. Publish code and non-sensitive traces, not merely aggregate scores.

Comparisons

- Single-host serving when the chosen model fits.
- Split serving with a static route and an ordinary database.
- Split serving with the same route and ledger accounting.
- Adaptive routing under the same hardware, traffic and failure schedule.
- Added replicas versus added sequential stages; these are not equivalent interventions.

Start with two physical machines, then four and eight only if earlier gates pass. Test same-LAN and deliberately constrained links, heterogeneous memory limits, provider joins/leaves, one failing stage, invalid outputs, delayed receipts and unavailable evidence. Repeat scenarios across fixed seeds and independent runs. Bootstrap or repeated-run intervals must respect clustered workloads rather than treating correlated tokens as independent samples.

Metrics

Report completed jobs meeting quality/deadline, p50/p95 end-to-end latency, time to first token, inter-token latency, aggregate throughput, peak memory per worker, bytes per request, restart/recovery cost, audit overhead, ledger cost and provider concentration. Measure actual wall-power consumption if making energy claims; do not infer energy savings from nominal FLOPs alone.

Model quality uses an independently held-out evaluation, paired with the reference implementation and a predeclared acceptable regression. Numerical tolerance must be chosen for the runtime before inspecting outputs. Adaptive scheduling is successful only if improvement survives the added audit and ledger costs and does not merely select easier requests.

Stop conditions

Do not expand to a public network if trained-model output equivalence is unexplained, dishonest results cannot be bounded under the stated trust model, evidence cannot be retrieved, or privacy requirements are unmet. Abandon the ledger for the target use case if independently controlled participants do not need it or its overhead eliminates the measured benefit. If network latency defeats interactive serving, narrow the workload to batch inference rather than invent a speedup claim.

9. Delivery gates and open decisions

G0 - Existing local demonstrator. Two local worker processes execute the untrained fixture; the registry checks signers, order and hash continuity. This gate establishes a test harness only. It is implemented.

G1 - Trained-encoder equivalence. Implemented for the Apache-2.0 all-MiniLM-L6-v2 encoder at a pinned revision. Two CPU workers execute three layers each, and normalized sentence embeddings are compared with the original full-model forward. Sixty observations across three workload shapes matched exactly locally. This is an embedding encoder, not autoregressive language generation.

G2 - Remote federation. Replace IPC with authenticated transport and workload manifests. Run two independently operated physical machines, enforce message limits and deadlines, and measure activation traffic and failures. Clarify coordinator authority and key custody. Not implemented.

G3 - Adaptive placement. Gather trustworthy workload-specific observations, compare deterministic baselines, then shadow an adaptive policy. Introduce limited canaries only after predeclared gains and non-regression conditions hold. Do not train on requester content without authorization. Not implemented.

G4 - Verification and accounting. Specify adversarial assumptions, audit selection, retention and dispute handling; test malicious nodes and missing evidence. Introduce nonredeemable credits first. Payment contracts require a separate design and audit. Not implemented.

G5 - Governed protocol evolution. Implement proposal records, independent approval, timelocks if justified, version pinning and routing rollback. Consider dedicated chain infrastructure only after publishing evidence that an existing settlement layer is inadequate. Not implemented.

Decisions still requiring evidence

Checkpoint and licensing; supported hardware and precision; centralized versus federated coordination; audit trust and cost; evidence retention; target workload economics; evaluator independence; identity admission; confidentiality requirements; and the legal/operational structure of any future paid service. This draft makes no claim that these questions are solved.

The recommended next engineering step is G2: a controlled two-machine private benchmark using the existing trained encoder. This tests the compute premise before adding economic or consensus complexity. The project's strongest portfolio value is a reproducible investigation with explicit falsification criteria, even if the final conclusion is that some workloads do not benefit from a blockchain.

10. Evidence, reproducibility and references

Existing evidence only

The initial untrained fixture is preserved at commit `a5d9e91` and in `evaluation/demo.json`. Those observations are separate from the trained-model experiment.

The v0.2 benchmark uses all-MiniLM-L6-v2 [6], revision `1110a243fdf4706b3f48f1d95db1a4f5529b4d41`. On 30 September 2026, Python 3.10.4, PyTorch 2.10.0 CPU and Transformers 4.57.6 ran on Windows with one PyTorch thread per process. All sixty observations across three workload shapes matched the full-model reference exactly. Thirty JavaScript/contract tests and thirteen Python tests passed locally. This establishes neither cryptographic correctness nor model-quality improvement.

`evaluation/embedding-benchmark.json` contains raw timings and settings.

`evaluation/failures.json` records one crash, stall and invalid-input observation each. See `docs/benchmark.md` for boundaries and caveats. Remote execution, adaptive scheduling, governance, training, payments and inference proofs remain unimplemented.

After dependency installation, reproduce with `npm run benchmark` and `python -m benchmark.failures`. Both rewrite observations. `npm run demo` is the separate untrained fixture. CI uses a three-repeat smoke benchmark rather than replacing the published twenty-repeat local measurements.

Primary references

[1] Borzunov et al. (2023). Distributed Inference and Fine-tuning of Large Language Models Over The Internet. NeurIPS 2023. <https://arxiv.org/abs/2312.08361>

[2] Arun et al. (2025). Verde: Verification via Refereed Delegation for Machine Learning Programs. <https://arxiv.org/abs/2502.19405>

[3] Bittensor. Official documentation: roles of miners, validators and subnet creators. Accessed 30 September 2026. <https://www.bittensor.com/docs>

[4] Douillard et al. (2023; revised 2024). DiLoCo: Distributed Low-Communication Training of Language Models. <https://arxiv.org/abs/2311.08105>

[5] Ethereum. Oracles: the boundary between off-chain computation and on-chain state. Accessed 30 September 2026. <https://ethereum.org/developers/docs/oracles/>

[6] Sentence Transformers. all-MiniLM-L6-v2 model card, Apache-2.0; pinned checkpoint revision recorded above. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

Authorship and status

Prepared by Kilian Codaccioni with AI-assisted research and drafting. This is a research proposal, not a peer-reviewed security proof, investment memorandum or novelty claim. No employer/client materials or upstream results are presented as original work. The project name has not undergone trademark clearance.